color_saved_display.v, explained part by part

Shows the memory saved by the compression, as a percentage, on the DE2-115 seven-segment displays. Prepared by Dev Vijay for the project review.

1. What the module does

The compression engine reports **total_bits**, the size of the compressed picture. This module compares it with the size of the original 24-bit colour picture and shows the **% saved** on HEX3 to HEX0, for example **96 4** = 96.4 %. While the engine is still working (done = 0) it shows dashes. It is purely combinational: no clock, no registers.

```
original size  = 320 x 320 pixels x 24 bits = 2,457,600 bits
compressed (%) = total_bits / 2,457,600 x 100
saved (%)      = 100 - compressed (%)
```

From bit count to digits

2. The code, part by part

Part A - Ports (lines 18-22)

Lines 18-22

```
module color_saved_display (
    input  wire        done,          // engine finished
    input  wire [23:0] total_bits,    // compressed size of Y + Cb + Cr in bits
    output wire [6:0]  HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7
);
```

- **done**: 1 when the engine has finished the picture; until then the display shows dashes.
- **total_bits**: 24-bit compressed size from color_dct_engine.
- **HEX0 to HEX7**: 7 bits each, one per segment. All 8 are outputs so every display has a defined value (unused ones are switched off).

Part B - Calculating the percentage without a divider (lines 24-28)

Lines 24-28

```
    // ---------- 1. Percentages in tenths (e.g. 964 = 96.4 %) ----------
    wire [39:0] prod          = {16'd0, total_bits} * 40'd3413 + 40'd4194304;  // + 2^22 rounds
    wire [39:0] comp_x10_full = prod >> 23;
    wire [9:0]  comp_x10      = (comp_x10_full > 40'd1000) ? 10'd1000 : comp_x10_full[9:0];
    wire [9:0]  saved_x10     = 10'd1000 - comp_x10;
```

We want the compressed size in tenths of a percent: total_bits x 1000 / 2,457,600. Dividing by 2,457,600 would need a large hardware divider, so the code multiplies by a fraction with a power-of-two bottom instead:

```
1000 / 2,457,600 = 0.00040690
3413 / 2^23      = 3413 / 8,388,608 = 0.00040686   (error under 0.01 %)

so  compressed_x10 = (total_bits x 3413) >> 23
```

- **+ 4,194,304** (= 2^{22}, half of 2^{23}) before shifting rounds to the nearest value instead of always rounding down.
- 40-bit width: total_bits (up to 2^{24}) x 3413 needs about 36 bits; 40 leaves room.
- Line 27 limits the result to 1000 (100 %) for safety; line 28 gives **saved_x10 = 1000 - compressed_x10**.

Worked example

Step	Value
total_bits	100,000
x 3413 + 4,194,304	345,494,304
>> 23 (divide by 8,388,608)	41 -> compressed = 4.1 %
1000 - 41	959 -> saved = 95.9 %
Digits	tens 9, ones 5, tenths 9 -> display shows 95 9

Part C - Splitting into digits (lines 30-33)

Lines 30-33

```
    // ---------- 2. Split into digits ----------
    wire [3:0] d_tens   = saved_x10 / 100;
    wire [3:0] d_ones   = (saved_x10 / 10) % 10;
    wire [3:0] d_tenths = saved_x10 % 10;
```

- saved_x10 is 0..1000. Dividing by 100 gives the tens digit, (/10) % 10 the ones digit, % 10 the tenths digit.
- These divide by **constants**, so Quartus builds them as small fixed logic, not a general divider.

Part D - Digit to 7-segment pattern (lines 35-54)

Lines 35-54

```
    // ---------- 3. Digit -> 7-segment pattern (segments are ON when 0) ----------
    localparam [6:0] SEG_BLANK = 7'b1111111;
    localparam [6:0] SEG_DASH  = 7'b0111111;

    function [6:0] seg;
        input [3:0] d;
        case (d)
            4'd0: seg = 7'b1000000;
            4'd1: seg = 7'b1111001;
            4'd2: seg = 7'b0100100;
            4'd3: seg = 7'b0110000;
            4'd4: seg = 7'b0011001;
            4'd5: seg = 7'b0010010;
            4'd6: seg = 7'b0000010;
            4'd7: seg = 7'b1111000;
            4'd8: seg = 7'b0000000;
            4'd9: seg = 7'b0010000;
            default: seg = SEG_BLANK;
        endcase
    endfunction
```

How the 7 bits map to segments

```
  segment bits:     --a--          bit 0 = a (top)        bit 4 = e (lower left)
                   |     |         bit 1 = b (upper right) bit 5 = f (upper left)
                   f     b         bit 2 = c (lower right) bit 6 = g (middle)
                   |--g--|         bit 3 = d (bottom)
                   e     c
                   |     |         Example: 0 = 1000000 -> all on except g
                    --d--                   1 = 1111001 -> only b and c on
```

- The DE2-115 displays are **active low**: a segment lights when its bit is **0**.
- **SEG_BLANK** = all 1s (all off); **SEG_DASH** = only g (bit 6) on, a minus sign.
- The function is a lookup table: digit in, 7-bit pattern out.

Part E - Driving the displays (lines 56-64)

Lines 56-64

```
    assign HEX3 = !done ? SEG_DASH : ((d_tens == 4'd0) ? SEG_BLANK : seg(d_tens));
    assign HEX2 = !done ? SEG_DASH : seg(d_ones);
    assign HEX1 = !done ? SEG_DASH : SEG_BLANK;
    assign HEX0 = !done ? SEG_DASH : seg(d_tenths);

    assign HEX4 = SEG_BLANK;
    assign HEX5 = SEG_BLANK;
    assign HEX6 = SEG_BLANK;
    assign HEX7 = SEG_BLANK;
```

- While **done = 0** all four show dashes, so nobody reads a half-finished number.
- HEX3 is blanked when the tens digit is 0 (e.g. 8.5 % shows as ' 8 5', not '08 5').
- **HEX1 is always blank** and acts as the decimal point, because DE2-115 displays have no dot segment.
- HEX7 to HEX4 are switched off.

3. Questions the professor may ask

Q: Why compare with 24 bits per pixel when the memory stores 16?

The original picture is a normal 24-bit colour image (8 bits each for R, G, B); RGB565 is only how we store it inside the FPGA to save memory.

Q: Why not use a divider?

A 24-bit divider is large and slow. Multiplying by 3413 and shifting by 23 gives the same result within 0.01 % using one multiplier.

Q: Why are the segments active low?

On the DE2-115 each segment's LED is connected so that a 0 on the FPGA pin lets current flow and lights it.

Q: Where is the decimal point?

The DE2-115 displays have no dot, so HEX1 is left blank as a gap: '96 4' means 96.4 %.

Q: Is total_bits the real file size?

It is the size counted with JPEG-style run-length symbols of fixed 8 bits. A real JPEG file with Huffman coding would be a further 20-30 % smaller, so the true saving would be even higher.